STUDY GUIDE · 02 OF 03

MockMate
Feature by Feature

Nineteen features, one chapter each. For every feature: what it does and why, the files involved, a flow diagram, the real code, the edge cases it handles, how to test it yourself, and how to explain it in an interview.

Auth · Resume parsing · Start + credits · Voice · Scoring · Report · History · Payments · Queue · Redis · Logging · Tests · Deploy

READ FIRST
01 Main Guide (big picture)

THEN
03 Interview Questions

CODE
github.com/chandandev6/Ai_interview_Agent

TIP
Keep the code open next to this PDF

Contents

F1	Google sign-in	Firebase ID token → JWT cookie
F2	Email + password accounts	bcrypt, verify link, forgot password
F3	Staying logged in (client session)	current-user, ProtectedRoute, 401
F4	Resume upload and parsing	multer, pdf-parse, background job
F5	Starting an interview	idempotency, credits, AI questions
F6	Voice: speaking and listening	browser first, Groq fallback
F7	The live interview round	phases, timer, refresh-safe
F8	Answer evaluation	claim-first, rubric, scores
F9	Finish and the report	job, weighted score, AI summary
F10	Report page and PDF export	Step3report, jspdf
F11	History and analytics	pagination, $facet, cache
F12	Credits ledger	transactions, passbook
F13	Payments with Razorpay	order, verify, webhook
F14	The AI client	providers, prompts, retries
F15	Background queue and cleanup	BullMQ, locks, self-healing
F16	Redis: rate limits and cache	fail open
F17	Config, validation and errors	env.js, validate(), AppError
F18	Logging, monitoring, health	pino, request IDs, /ready
F19	Tests, CI and deployment	Vitest, Actions, Vercel, Render

F1 Google sign-in

One click "Continue with Google". The browser gets a signed token from Google (via Firebase), our server checks that token, and then gives the browser *our own* session cookie.

Why it is built this way

The browser could just send "my email is x@gmail.com" — but anyone can send that. A **Firebase ID token** is signed by Google. Our server verifies the signature with Firebase Admin, so the email inside is trustworthy. After that we stop using Firebase and issue our own JWT, so the rest of the app doesn't care how you logged in.

Files

File	Role
`client/src/utils/firebase.js`	Firebase app + `GoogleAuthProvider`
`client/src/pages/Auth.jsx`	`signInWithPopup` → `getIdToken()` → `googleLogin(idToken)`
`server/src/config/firebaseAdmin.js`	Firebase Admin with only the project id (enough to verify tokens)
`server/src/modules/auth/auth.service.js`	`loginWithGoogle`: verify token, require verified email
`server/src/modules/user/user.service.js`	`findOrCreateGoogleUser`
`server/src/modules/auth/auth.controller.js`	`setAuthCookie`: JWT in an httpOnly cookie for 7 days

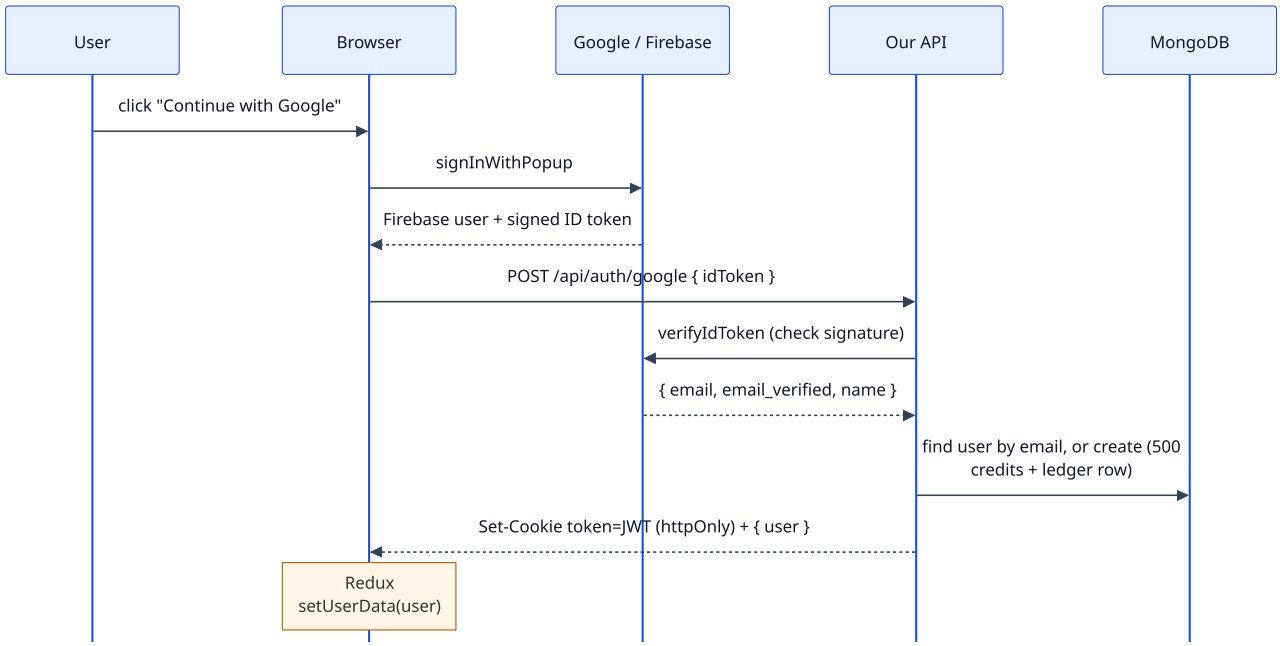


Figure F1.1 — Google login. After this, only our cookie matters.

The code

server/src/modules/auth/auth.service.js

```
export const loginWithGoogle = async (idToken) => {
    let decoded;
    try {
        decoded = await firebaseAuth.verifyIdToken(idToken);
    } catch {
        throw new AppError(401, "Invalid Google token");
    }
    if (!decoded.email || !decoded.email_verified) {
        throw new AppError(401, "Google account email not verified");
    }
    return findOrCreateGoogleUser({
        email: decoded.email.toLowerCase(),
        name: decoded.name || decoded.email.split("@")[0],
    });
};
```

server/src/modules/auth/auth.controller.js + config/cookie.js

```
const setAuthCookie = (res, user) => {
    res.cookie("token", genToken(user._id), { ...cookieOptions, maxAge: SEVEN_DAYS });
};
// cookieOptions: httpOnly: true, secure: isProd, sameSite: isProd ? "none" : "strict"
```

Edge cases handled

Situation	What happens
Token is fake / expired	`verifyIdToken` throws → 401 "Invalid Google token".
Google email not verified	401 — we only accept verified emails.
Email exists as an **unverified** password account	Could be an attacker who registered your email. The real owner just proved ownership with Google → we mark verified and **delete the unverified password**, so the attacker can't log in.
Brand-new user	Created with `isVerified: true`, default 500 credits, and a `signup_bonus` ledger row.
Email casing `Abc@Gmail.com`	Lower-cased everywhere; the schema also lowercases.

TEST IT YOURSELF

Log in with Google, then open DevTools → Application → Cookies: you'll see `token` with **HttpOnly** ✓. In the console, `document.cookie` will *not* show it — that's the point.

SAY IT IN AN INTERVIEW

"The client uses Firebase only for the Google popup. It sends the Firebase ID token to my server, which verifies the signature with Firebase Admin and requires a verified email. Then I issue my own JWT in an httpOnly cookie, so the rest of the app has one session mechanism for both Google and password users."

F2 Email + password accounts

Register → verify email → log in. Plus "forgot password". Built by hand with bcrypt and random link tokens (not Firebase), to learn how it works.

Endpoints

Endpoint	What it does	Limiter
POST `/api/auth/register`	Create account (unverified), email a link. No login yet.	auth 10/15 min
POST `/api/auth/verify-email`	Token from the link → verified → logged in (cookie).	—
POST `/api/auth/login`	Email + password → cookie. Unverified → 403 `EMAIL_NOT_VERIFIED`.	auth
POST `/api/auth/resend-verification`	New link (always the same reply).	auth
POST `/api/auth/forgot-password`	Reset link by email (always the same reply).	auth
POST `/api/auth/reset-password`	Token + new password → logged in.	auth
POST `/api/auth/logout`	Clear the cookie.	—

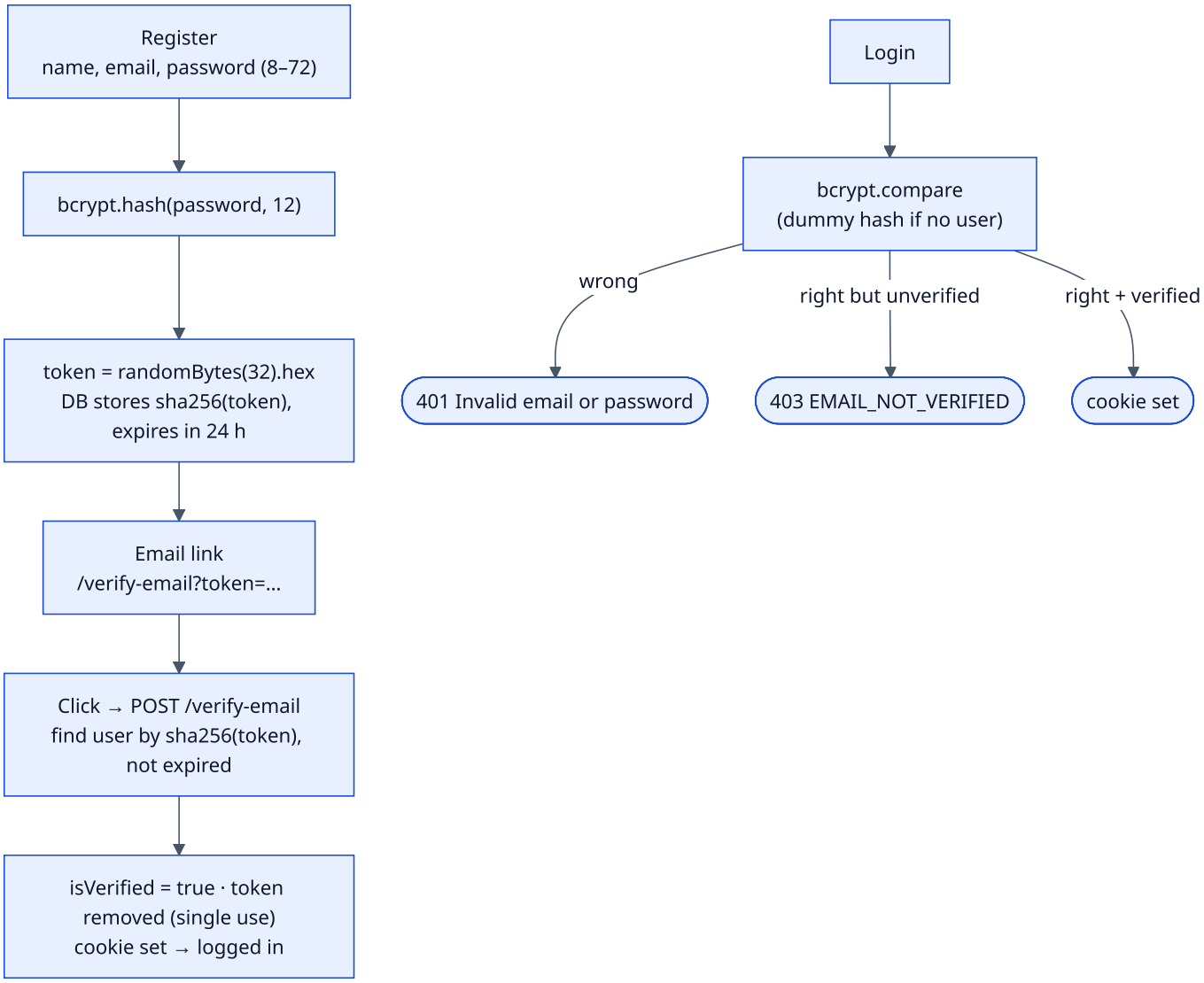


Figure F2.1 — Register/verify and login.

The important code

`server/src/modules/auth/auth.service.js`

```
const BCRYPT_ROUNDS = 12; // ~0.2 s per hash — fine for a user, very slow for an attacker
const DUMMY_HASH = bcrypt.hashSync("dummy-password-for-timing", BCRYPT_ROUNDS);
const hashToken = (token) => crypto.createHash("sha256").update(token).digest("hex");

export const loginWithPassword = async ({ email, password }) => {
    const user = await User.findOne({ email }).select("+password");
    // no user? still run a compare, so the response time doesn't reveal it
    const isMatch = await bcrypt.compare(password, user?.password || DUMMY_HASH);
    if (!user || !user.password || !isMatch) throw new AppError(401, "Invalid email or password");
    // only AFTER the right password do we reveal "not verified"
    if (!user.isVerified) throw new AppError(403, "Please verify your email first...",
"EMAIL_NOT_VERIFIED");
    return toSafeUser(user);
};
```

Security ideas in this feature

Hash the link token

The email contains the plain token; the DB stores only its SHA-256. If the database leaks, nobody can build a working link. SHA-256 (fast) is fine here because the token is 256 random bits — impossible to guess, unlike a password.

No account enumeration

Same message for unknown email and wrong password; dummy bcrypt compare; forgot-password always answers "if an account exists..." and sends the email **without await** (SMTP takes ~1 s — waiting only for real users would leak by timing).

Single use + expiry

Verify links last 24 h, reset links 1 h. After use the token fields are removed, so the same link can't be used twice.

Secret fields hidden

`password` and tokens have `select: false`; `toSafeUser()` strips them before any response.

Edge cases

Situation	Result
Register with an email that is a Google-only account	409 with a hint: use Google, or "Forgot password" to set one.
Register again before verifying	Allowed: new name/password, new link. Only the real inbox owner can verify.
Two registrations at the same moment	Unique email index → second gets E11000 → 409.
Password longer than 72 chars	Rejected (bcrypt silently ignores bytes after 72).
Reset link used	Sets password, marks email verified (clicking the email proves ownership), logs in.
Email can't be sent (SMTP down)	Register → 502 with "try Resend in a minute".

KNOWN GAP (WRITTEN IN THE PRD)

Resetting the password doesn't log out other devices: a JWT stays valid until it expires (7 days). Fix later with a `tokenVersion` on the user that `isAuth` checks.

SAY IT IN AN INTERVIEW

"Passwords are hashed with bcrypt at cost 12. Verification and reset links use 32 random bytes; only the SHA-256 hash is stored, links are single-use and expire. I avoid user enumeration with identical messages and a dummy bcrypt compare so timing is the same."

F3 Staying logged in (client session)

The JWT cookie is invisible to JavaScript, so how does React know who is logged in? It asks the server once when the app opens.

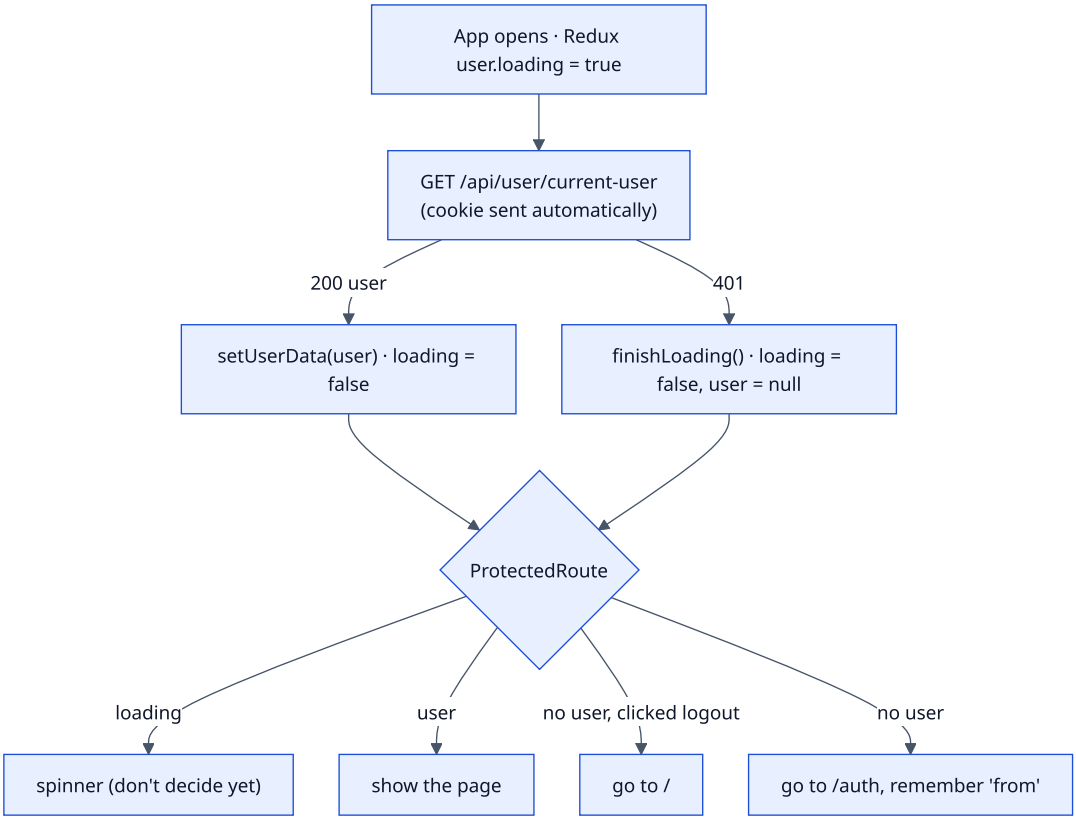


Figure F3.1 — Session on the client.

Three small details that fix real bugs

1. `loading: true` **at start.** Without it, on refresh the user is `null` for a moment and ProtectedRoute would kick a logged-in user to the login page.
2. **App.jsx does not dispatch** `null` **on 401.** If that slow response arrived *after* the user just logged in, it would log them out again (race). It only calls `finishLoading()`.
3. **The 401 interceptor ignores** `/api/auth/*` **and current-user.** A wrong password (401) is not "session expired". For other calls, a 401 clears the user and ProtectedRoute sends them to login.

client/src/api/axios.js

```
const api = axios.create({ baseURL: serverUrl, withCredentials: true });
const IGNORE_401 = ["/api/auth/", "/api/user/current-user"];

api.interceptors.response.use((response) => response, (error) => {
    const url = error.config?.url || "";
    const isSessionExpired = error.response?.status === 401 && !IGNORE_401.some((p) =>
url.startsWith(p));
    if (isSessionExpired && store.getState().user.userData) store.dispatch(setUserData(null));
    return Promise.reject(error); // the component's catch still runs
});
```

On the server, `GET /api/user/current-user` is cached in Redis for 60 s (it runs on every page load). Whenever credits or verification change, the cache key is deleted so the next call is fresh.

> **PROTECTEDROUTE IS ONLY UX**
>
> Hiding a page in React is not security — anyone can call the API directly. Real protection is `isAuth` on the server plus `userId` in every query.

F4 Resume upload and parsing

Optional. The user drops a PDF; the AI extracts skills, projects and an experience summary, which later make the questions personal ("Tell me about the caching in your Drishti project").

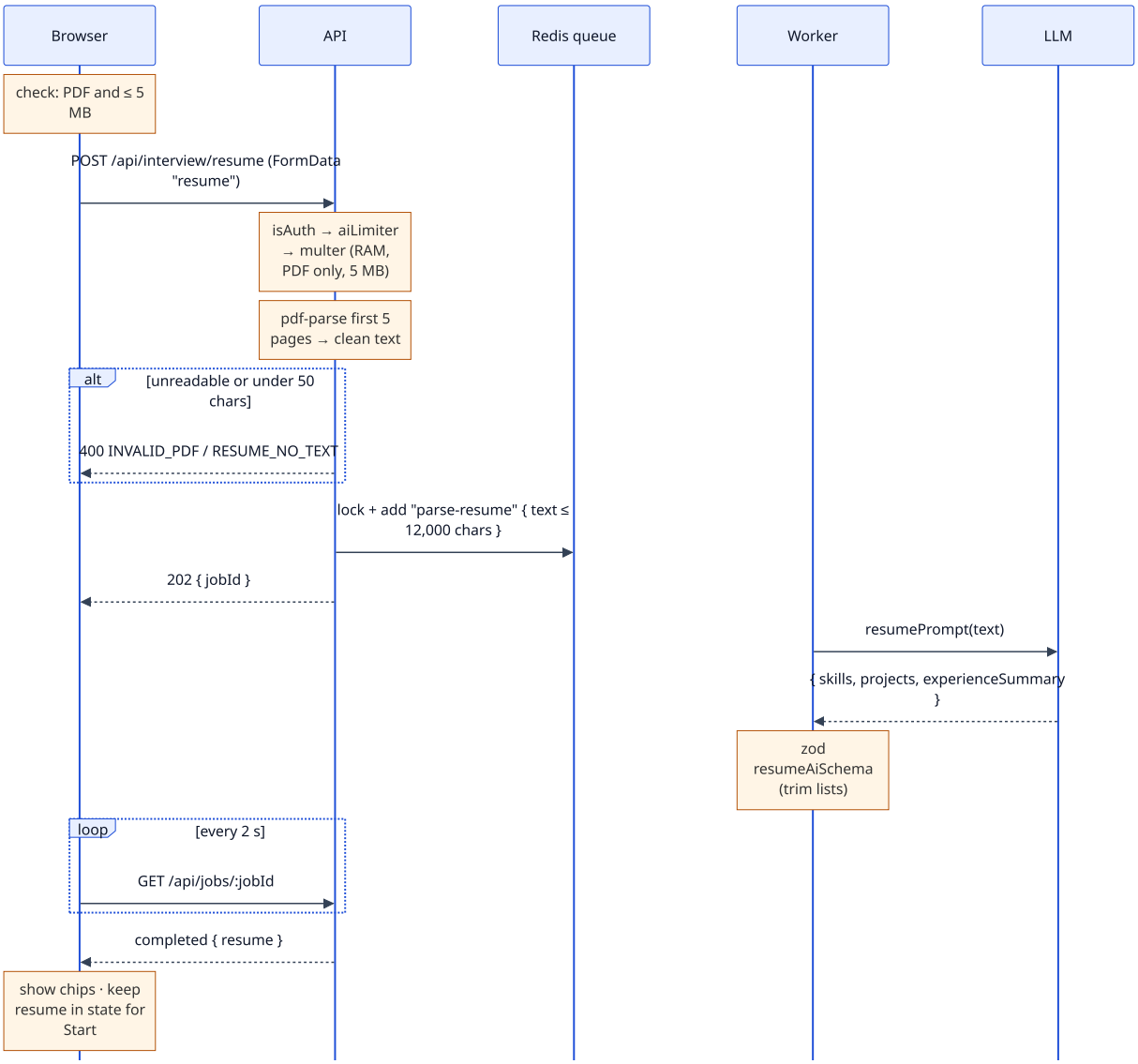


Figure F4.1 — Fast checks in the API, slow AI in the worker.

Why split it like this

- **PDF → text in the API** takes milliseconds and lets a bad file fail *immediately* with a clear 400. The `mimetype` is just a label from the browser; the real check is "can pdf-parse read it?".
- **Only text goes to Redis** (max 12,000 chars), not a 5 MB PDF — Redis memory is small.
- **Nothing is stored except the AI summary** — the PDF and full text are thrown away (privacy, small documents). The summary is saved inside the interview when it starts.
- **The AI's reply is untrusted**: `resumeAiSchema` keeps at most 20 skills and 5 projects and limits string lengths.

server/src/modules/interview/interview.service.js

```
const RESUME_MAX_PAGES = 5;      // a 300-page PDF must not keep the server busy
const RESUME_MAX_CHARS = 12000;  // more text = more tokens = more cost and time
const RESUME_MIN_CHARS = 50;     // less = probably a scanned image, no text layer

export const requestResumeParse = async (userId, buffer) => {
    const text = await extractPdfText(buffer);  // throws 400 INVALID_PDF
    if (text.length < RESUME_MIN_CHARS) throw new AppError(400, "No text found in this PDF...",
"RESUME_NO_TEXT");
    const jobId = await enqueueAiJob(JOB.PARSE_RESUME, userId, { text: text.slice(0,
RESUME_MAX_CHARS) });
    return { jobId };
};
```

Edge case	Result
No file / wrong field name	400 `RESUME_MISSING`
.docx renamed to .pdf, broken PDF	400 `INVALID_PDF` (pdf-parse fails)
Scanned resume (image only)	400 `RESUME_NO_TEXT` with a clear hint
6 MB file	Blocked in the browser; multer also rejects (400)
User uploads again while one is running	429 `AI_JOB_RUNNING` (per-user lock)
Resume text says "ignore instructions..."	It's inside `<resume>` tags as data; the prompt says never follow it

F5 Starting an interview

The most carefully ordered function in the project: it spends money (credits) and calls an AI that can fail. The order of steps is the whole trick.

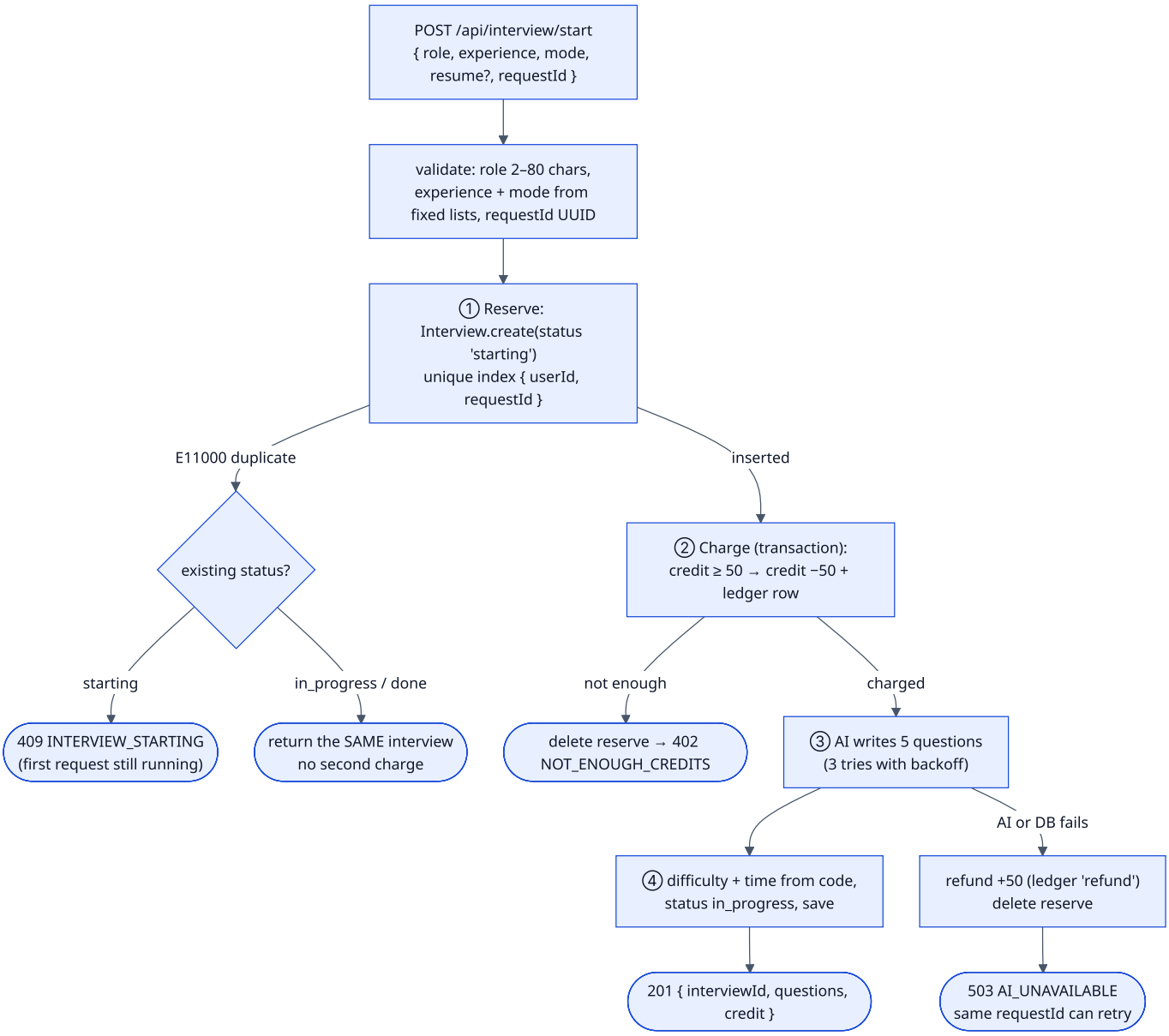


Figure F5.1 — Reserve → charge → AI → save, with a refund path.

Why each step is where it is

- **Reserve first:** a double-click sends two requests with the *same* `requestId`. Only one insert can succeed (unique index), so the second never reaches the charge step.
- **Charge before AI:** don't spend AI money for a user who can't pay.
- **Atomic charge:** `{ credit: { $gte: 50 } }` + `$inc: -50` in one query; credit can never go negative even with 10 parallel requests.
- **Refund on any failure after charging** — and delete the reserve so "Try again" works.
- **The browser keeps the same requestId** if the network failed (no answer from server) or the server said `INTERVIEW_STARTING`; for any other server answer it makes a new one.

server/src/modules/credit/credit.service.js

```
export const chargeCredits = async (userId, amount, { interviewId } = {}) => {
    const user = await mongoose.connection.transaction(async (session) => {
        const user = await User.findOneAndUpdate(
            { _id: userId, credit: { $gte: amount } },  // check ...
            { $inc: { credit: -amount } },               // ... and subtract, in ONE operation
            { returnDocument: "after", session },
        );
        if (!user) return null;
        await Transaction.create([{ userId, type: "interview", amount: -amount, balanceAfter:
user.credit, interviewId }], { session });
        return user;
    });
    if (user) await clearCache(cacheKeys.user(userId)); // after commit
    return user;
};
```

Questions are designed for speaking

The questions prompt says: exactly 5, in the order easy, easy, medium, medium, hard; each answerable **out loud** in its time (no code writing); 1–2 sentences; match the experience level; if a resume is given, base at least 2 questions on the candidate's own projects or skills. The AI returns **only the text**; the code adds `difficulty` and `timeLimitSec` (60/90/120) from `QUESTION_PLAN`.

CRASH IN THE MIDDLE?

If the server dies after charging but before refunding, the interview stays in `starting`. The cleanup job (every 60 s) finds `starting` rows older than 10 minutes, refunds them **only if the ledger shows a charge and no refund**, then deletes them. Running cleanup twice can't refund twice.

SAY IT IN AN INTERVIEW

"Start is idempotent: the client sends a UUID and a unique index on userId + requestId makes a duplicate request return the first interview. Credits are charged with a conditional atomic update inside a MongoDB transaction that also writes a ledger row. If the AI fails after charging, I refund and delete the reserve; a cleanup job covers crashes. A test fires 10 parallel starts with credit for one — exactly one succeeds."

F6 Voice: speaking and listening

Voice is **hybrid**. If the browser can do it well (Chrome, Edge, Safari), the free Web Speech API does it in the browser. If not (Firefox, Brave, Opera…), the browser records / plays audio and our server asks **Groq**: **Whisper** turns speech into text and **Orpheus** turns text into a natural voice. Two hooks hide this — the interview components never know which path ran.

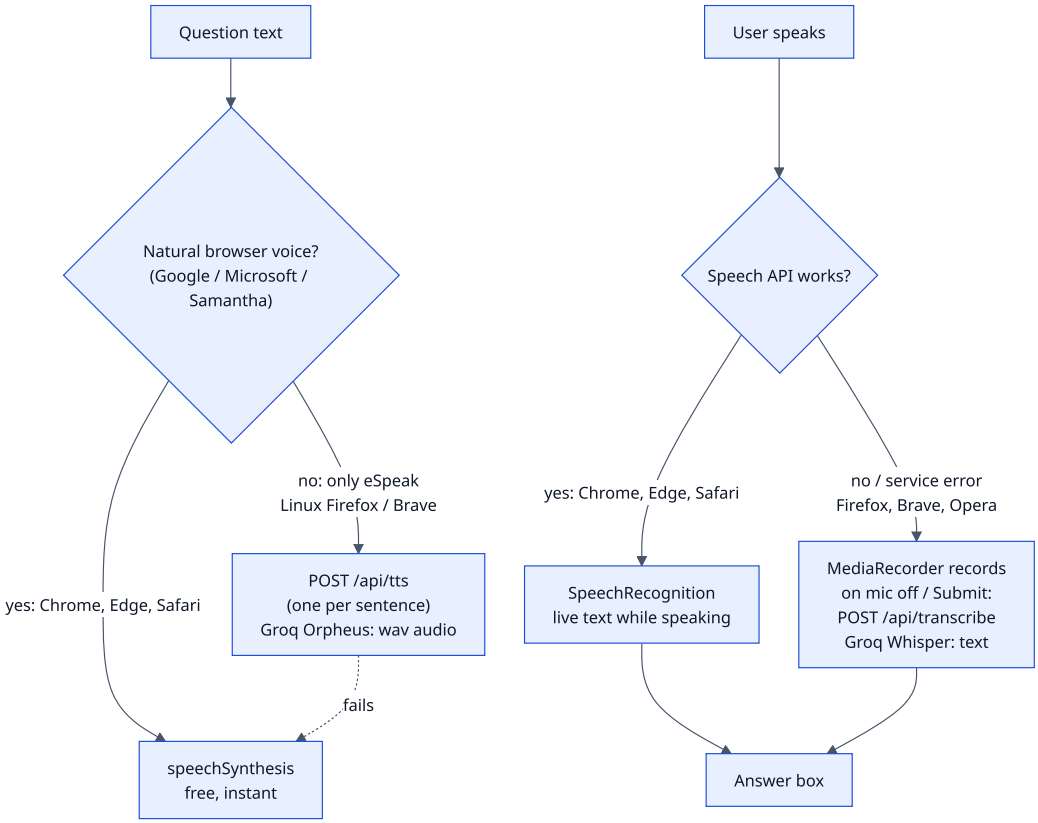


Figure F6.1 — Browser first (free), server only where the browser can't do it.

`useSpeechSynthesis(gender)` — AI speaks

- `speak(text)` returns a Promise that resolves when speech ends (or fails — the interview must never get stuck).
- Picks the best English voice by **score**: Google/Microsoft/Natural +4, robotic eSpeak variants (name contains "+") −4, gender match +2, en-US/GB/IN +1.
- `isNaturalVoice(best)` false → **server voice**: every sentence is requested in parallel from `/api/tts`, played one after another (sentence 2 downloads while 1 plays). Blob URLs are cached, so "Repeat question" costs no new request. Any failure → the rest is spoken by the browser voice.
- Waits up to 1 s for `voiceschanged` — voices load late, and an empty list must not mean "no good voice".
- Splits text into sentences (Chrome's Google voices stop after ~15 s), safety timeout for a missing `onend`, and a run counter so `cancel()` stops an older `speak()`.

`useSpeechRecognition(onFinalText)` — user speaks

- **Browser path:** `continuous` + `interimResults` (grey live text), `lang = "en-IN"`, restart when Chrome stops on silence.
- **Recorder path:** `MediaRecorder` (webm / ogg / mp4 — whatever the browser supports), on stop → `/api/transcribe` → text added. Silent recordings are **not sent** (an `AnalyserNode` watches the volume) because Whisper invents "Thank you." on silence.
- Auto-switch: a `network` / `service-not-allowed` error (Opera, Vivaldi, Linux Chromium, Safari with Siri off) flips to the recorder and is remembered until reload.
- Both paths: `stop()` returns a Promise that waits for the last text, so it reaches the answer before submit. Mic denied → "please type".

Files

File	Job
`client/src/hooks/useSpeechSynthesis.js`	voice scoring, browser vs server voice, audio cache
`client/src/hooks/useSpeechRecognition.js`	browser hook + recorder hook, one shared interface (`live`, `transcribing`)
`client/src/api/tts.api.js`, `transcribe.api.js`	upload audio / download wav (`responseType: "blob"`)
`server/src/modules/tts/`, `modules/transcribe/`	routes: `isAuth → limiter → validate / multer`; 503 `VOICE_UNAVAILABLE` on Groq errors
`server/src/lib/ai/voice.js`	Groq calls: `transcribeAudio` (Whisper, multipart) and `textToSpeech` (Orpheus, wav bytes)

client/src/hooks/useSpeechSynthesis.js (core of speak)

```
const voice = pickVoice(await loadVoices(), gender);
if (isNaturalVoice(voice)) await speakWithBrowser(parts, voice);   // free, instant
else await speakWithServer(parts, voice, isStale);              // Groq Orpheus

// server voice: ask for all sentences at once, play in order
const urls = parts.map((part) => getSpeechUrl(part, gender));   // cached Promise<blob URL>
for (let i = 0; i < parts.length; i++) {
    try { await playAudio(audio, await urls[i]); }
    catch { await speakWithBrowser(parts.slice(i), voice); return; }  // never get stuck
}
```

server/src/modules/tts/tts.controller.js

```
// not JSON — raw audio bytes (base64 in JSON would be 33% bigger). Errors are still JSON.
export const textToSpeech = async (req, res) => {
    const audio = await speak(req.body.text, req.body.gender);
    return res.type("audio/wav").send(audio);
};
```

REAL BUG: "VOICE IS ROBOTIC AND MY SPEECH ISN'T WRITTEN"

On the live site the voice sounded weird and speech wasn't recognised. The console list of voices showed only **eSpeak** voices → the user was on **Firefox on Linux**, which has no speech recognition and only robotic voices. In **Brave** the API exists but Brave blocks Google's speech service, so the mic shows "Listening" and writes nothing. **First fix:** score voices, speak in sentences, Brave = "please type". **Real fix:** the browser can't do better, so the server does it — Whisper for listening, Orpheus for speaking — only in those browsers, so Chrome users still cost nothing.

Browser	Speaking (TTS)	Listening (STT)
Chrome / Edge	browser — natural Google / Microsoft voices	browser — live text
Safari	browser — Samantha / Daniel	browser (Siri off → recorder)
Firefox / Brave (Linux)	**server** — Groq Orpheus (hannah / troy)	**server** — recorder + Groq Whisper, text after mic off
Opera / Vivaldi	depends on installed voices	speech service fails → auto-switch to recorder

CAREFUL

- **Cost & limits:** Groq free tier limits audio too, so separate limiters: `voiceLimiter` 20/min and `ttsLimiter` 40/min per user — mic on/off or Repeat doesn't eat the AI answer limit.
- **Privacy:** audio is held in RAM (multer memory) and forgotten — never saved.
- **Orpheus terms:** must be accepted once in the Groq console, otherwise every call is 400 `model_terms_required` (and the browser voice is used).
- **Autoplay:** `audio.play()` works because the user already clicked "Start"; if a browser blocks it, play rejects → browser voice.

How to test

- Firefox / Brave: the question should sound human; DevTools → Network shows `POST /api/tts` per sentence. Speak, press Mic off → "Converting your voice to text..." → text appears.
- Chrome: no `/api/tts` or `/api/transcribe` calls at all.
- Tests: `server/test/tts.test.js`, `transcribe.test.js` (Groq mocked); client hook tests fake `MediaRecorder` and `HTMLMediaElement.play`.

SAY IT IN AN INTERVIEW

"Voice is browser-first: the Web Speech API is free and has zero server load, but Firefox has no speech recognition and Linux browsers only have robotic voices. So I detect capability — not the browser name — and fall back to the server: MediaRecorder plus Groq Whisper for listening, Groq Orpheus for speaking. Both hooks keep the same Promise-based API, so the interview flow just `await`s `speak()` and `stop()` and never knows which path ran."

F7 The live interview round

`Step2interview` shows the avatar and owns the "finish" logic. `QuestionRound` runs one question. The parent renders it with `key={index}`, so every new question is a brand-new component and all its state resets automatically.

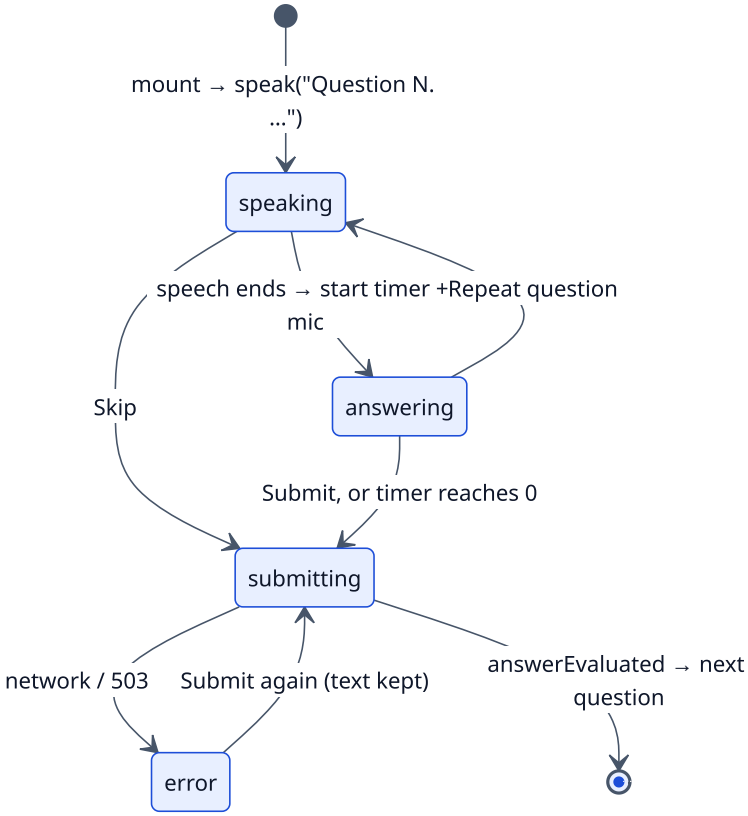


Figure F7.1 — The phases of one QuestionRound.

Things that make it robust

Problem	Solution
Timer callback / mic chunk use an old value of `answer` (stale closure)	Values are mirrored in refs (`answerRef`, `phaseRef`); async code reads the ref.
Tab in the background slows `setInterval`	`useCountdown` stores the end time and computes `endAt - Date.now()` every 250 ms.
Client could lie about time taken	Server caps `timeTakenSec` at the question's limit.
Page refresh mid-interview (50 credits already spent)	`sessionStorage` keeps `{ interviewId, avatar }`. On reload, `GET /api/interview/:id` → `interviewRestored` → continue at the first unscored question.
Browsers block audio without a user click	After a restore, `resumed: true` shows a "Continue" button first.
Submit succeeded on the server but the response was lost	Retry gets `409 ALREADY_ANSWERED` → the client reads the saved score with `GET /:id` and moves on.

React StrictMode runs effects twice in dev	The finish request is stored per attempt in a ref, so the second effect run reuses the same promise.
Accidentally closing the tab	`beforeunload` → browser asks "Are you sure?".

client/src/components/interview/QuestionRound.jsx

```
const askQuestion = async () => {
    await speak(`Question ${index + 1}. ${question.question}`);
    if (phaseRef.current !== "speaking") return; // skipped or unmounted meanwhile
    changePhase("answering");
    timer.start(question.timeLimitSec);
    mic.start();
};
```

KNOWN GAP

When all answers are in, `Step2interview` calls finish and auto-retries every 3 s on `INTERVIEW_FINISHING` / `ANSWER_BEING_EVALUATED` — with no upper limit. If the worker is down, the spinner never stops. A cap (~3 min) + Try again button is planned.

F8 Answer evaluation

`POST /api/interview/:id/answer { questionIndex, answer, timeTakenSec }` → 4 scores (0–10) and 2–3 sentences of feedback. This one runs inside the request because the user is waiting for it.

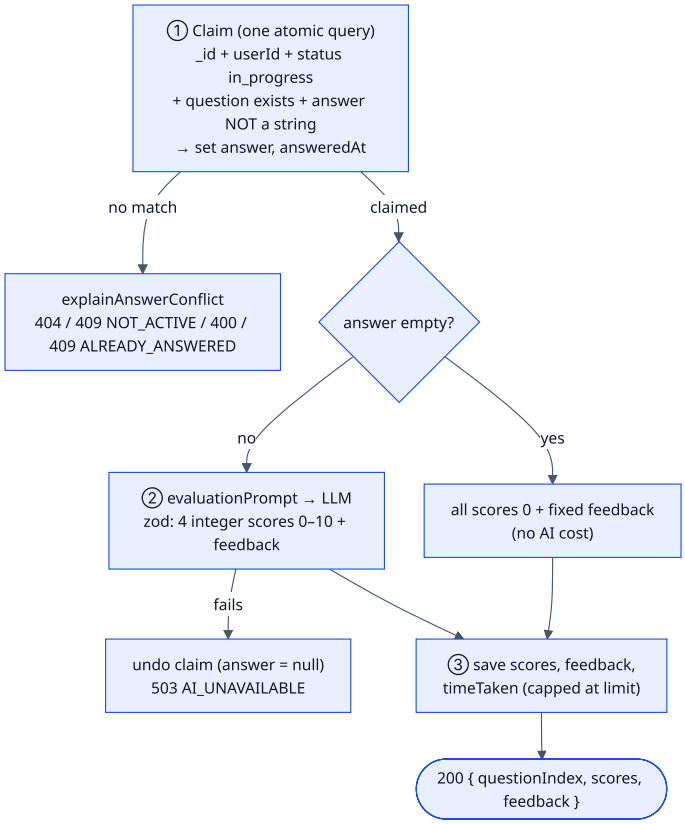


Figure F8.1 — Claim first, then the expensive work, then save.

The claim query

`server/src/modules/interview/interview.service.js`

```
const path = `questions.${questionIndex}`;
const interview = await Interview.findOneAndUpdate(
    {
        _id: interviewId,
        userId,                                          // ownership check
        status: "in_progress",
        [`${path}.question`]: { $exists: true },
        [`${path}.answer`]: { $not: { $type: "string" } }, // not answered yet
    },
    { $set: { [`${path}.answer`]: cleanAnswer, [`${path}.answeredAt`]: new Date() } },
    { returnDocument: "after" },
);
```

REAL BUG FOUND IN TESTING: `"QUESTIONS.3.ANSWER": NULL`

The first version used `{ "questions.3.answer": null }`. It matched **every** document. In MongoDB, a numeric path on an array means "index 3" *and also* "a field called 3 on each element"; that field is missing on every element, and missing counts as `null`. Fix: check `$not: { $type: "string" }` and that `question` exists at that index.

The rubric changes per mode

Mode	Communication	Content	Technical	Confidence
Technical	15%	30%	**45%**	10%
HR	**35%**	**35%**	10%	20%
Confidence	35%	20%	5%	**40%**

The evaluation prompt also tells the AI: the answer is a speech-to-text transcript (ignore spelling and punctuation), judge for the experience level and difficulty, and give 0–2 to anything off-topic, a repeat of the question, or an attempt to give the AI instructions. Scores are coerced to integers 0–10 by zod (the model sometimes sends `7.5` or `"8"`).

F9 Finish and the report

After the 5th answer, the client calls `POST /:id/finish`. The API only *claims* and queues; the worker builds the report.

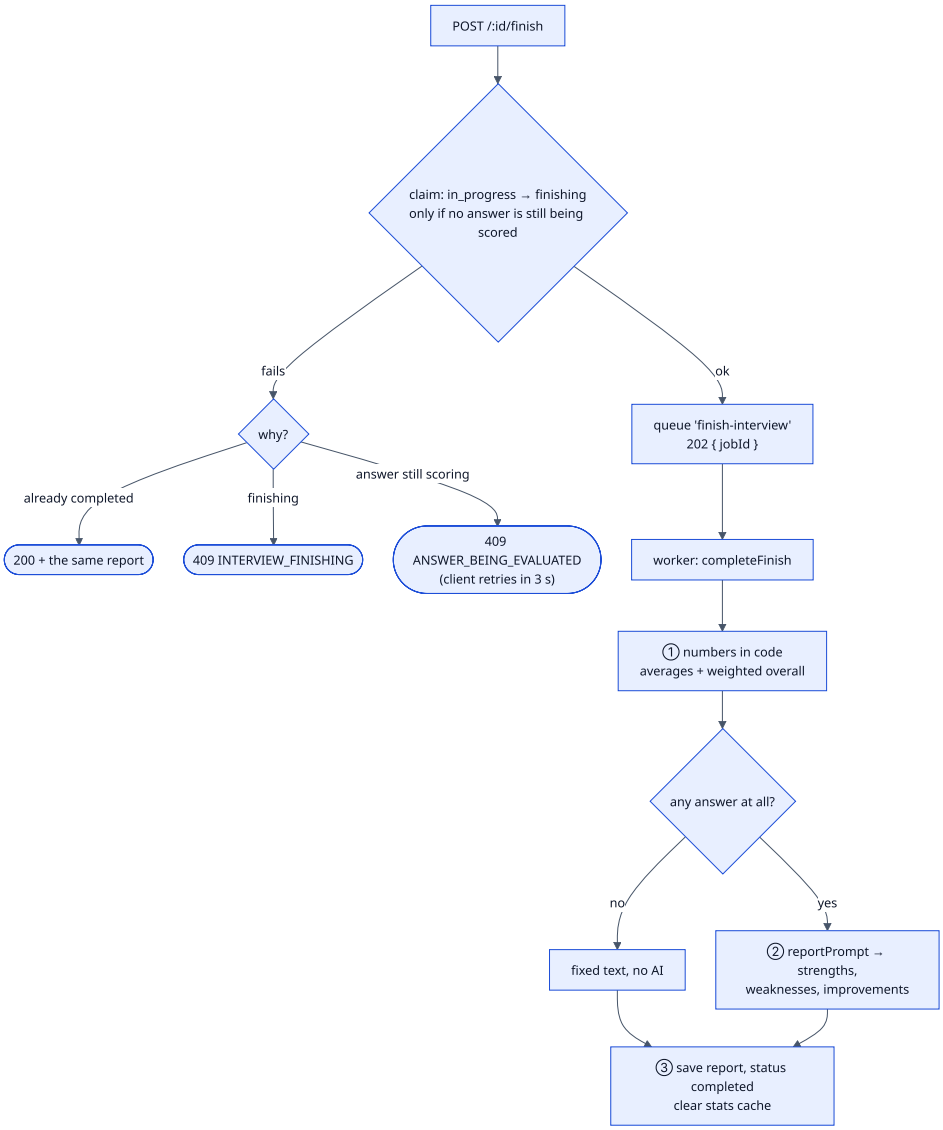


Figure F9.1 — Finish: claim in the API, work in the worker.

The score is maths, not AI

`server/src/modules/interview/interview.service.js`

```
const calculateScores = (questions, weights) => {
    const averages = Object.fromEntries(CRITERIA.map((key) => {
        const total = questions.reduce((sum, q) => sum + (q.scores?.[key] ?? 0), 0); // unanswered =
0
        return [key, Math.round((total / questions.length) * 10) / 10];        // e.g. 6.4
    }));
    // weighted average (0-10) × 10 → 0-100
    const overallScore = Math.round(CRITERIA.reduce((sum, key) => sum + averages[key] * weights[key],
0) * 10);
    return { averages, overallScore };
};
```

WORKED EXAMPLE (TECHNICAL MODE)

Averages: communication 6.2, content 6.4, technical 6.6, confidence 6.8.
Weighted: 6.2×0.15 + 6.4×0.30 + 6.6×0.45 + 6.8×0.10 = 0.93 + 1.92 + 2.97 + 0.68 = 6.50 → ×10 = **65**. This exact case was verified by hand during development.

Retry behaviour of the job

- The job has 3 attempts (backoff 2 s, 4 s). Between attempts the status stays `finishing`.
- Only the **last** failed attempt puts the interview back to `in_progress`, so the user can press Finish again.
- If the job is re-run after a crash and the interview is already `completed`, it returns at once (idempotent). If the status is something else, it throws `UnrecoverableError` — no point retrying.
- The save uses `{ status: "finishing" }` in its filter, so it can't overwrite an interview that moved to another state.

F10 Report page and PDF export

`/report/:id` shows the result. The PDF is generated **in the browser** — zero server load.

Report page (`Interviewreport` + `Step3report`)

- Right after finish, the interview is passed through router `state` (no extra request). On a refresh, it is loaded with `GET /api/interview/:id`.
- Animated score ring (/100), 4 skill bars, strengths / weaknesses / improvements, and an accordion with every question, answer, score and feedback.
- Skeleton while loading; "not found" and "not finished" states.

PDF (`utils/reportPdf.js`)

- `jspdf` + `jspdf-autotable` are ~400 kB, so they are **dynamically imported** only when "Download PDF" is clicked.
- Header with role and coloured score, skills table, lists, one table per question, page numbers.
- jsPDF's built-in fonts only know Latin-1; AI text contains - ’ “ … → replaced with simple characters, anything else removed.

client/src/utils/reportPdf.js

```
export const downloadReportPdf = async (interview) => {
    // separate chunks, downloaded only now
    const [{ jsPDF }, { autoTable }] = await Promise.all([import("jspdf"), import("jspdf-
autotable")]);
    const doc = new jsPDF({ unit: "mm", format: "a4" });
    ...
};
```

WHY DYNAMIC IMPORT MATTERS

Most users never click Download. A static import would make every report page load 400 kB more JavaScript. With `import()`, Vite splits it into its own file that is fetched on demand.

F11 History and analytics

`/history` lists past interviews (9 per page, mode tabs, "Load more") and shows analytics on top: KPI tiles, a score-trend chart, skill bars and the weakest skill.

History: fast pagination

server/src/modules/interview/interview.service.js

```
const HISTORY_FIELDS = "role experience mode status creditsUsed report.overallScore createdAt
completedAt";

export const getHistory = async (userId, { page, limit, mode }) => {
    const filter = { userId, status: { $ne: "starting" } };  // a reserve is not a real interview
    if (mode) filter.mode = mode;
    const [items, total] = await Promise.all([               // both queries in parallel
        Interview.find(filter).sort({ createdAt: -1 }).skip((page - 1) * limit).limit(limit)
            .select(HISTORY_FIELDS).lean(),                      // light fields, plain objects
        Interview.countDocuments(filter),
    ]);
    return { items, pagination: { page, limit, total, totalPages: Math.ceil(total / limit) } };
};
```

- **Index** `{ userId: 1, createdAt: -1 }` serves both the filter and the sort.
- **Projection**: no questions/resume (the heaviest fields) in the list.
- `.lean()`: plain objects instead of Mongoose documents — faster, less memory.
- **limit ≤ 50** (zod), so nobody can ask for the whole collection.

Analytics: one aggregation with `$facet`

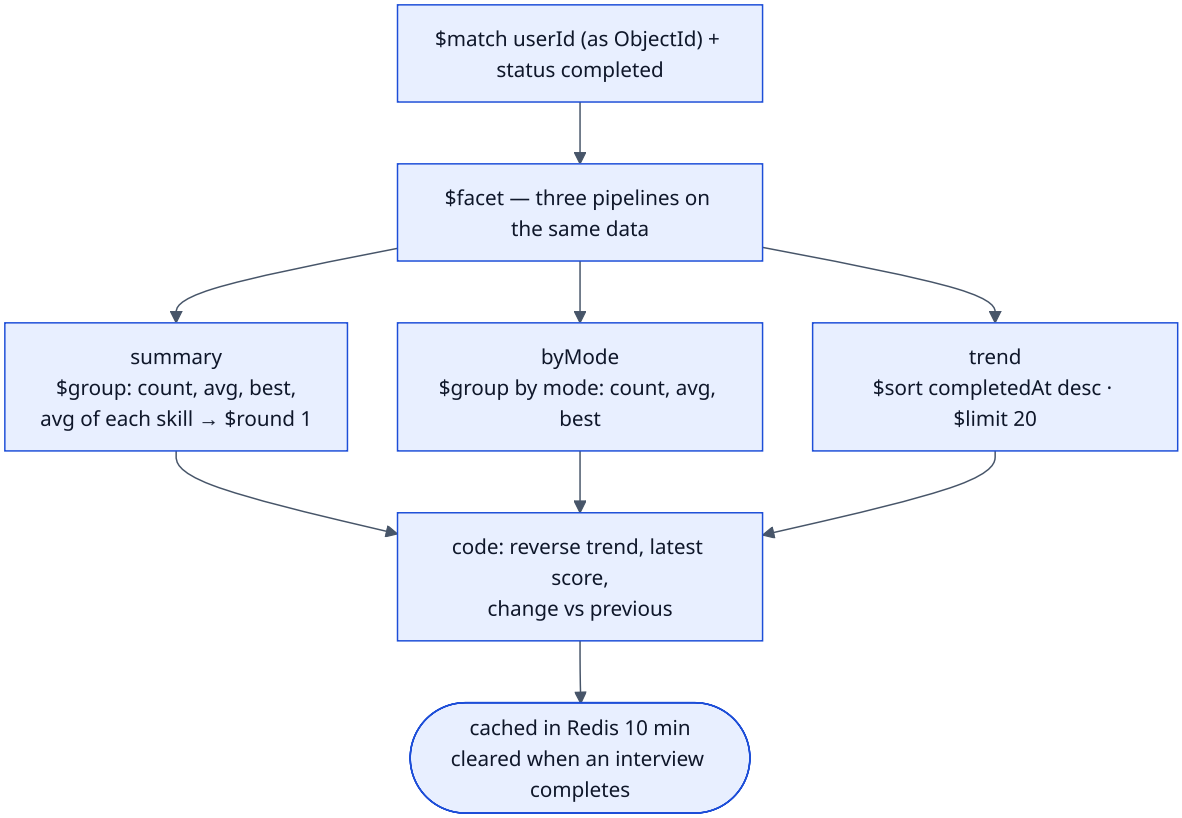


Figure F11.1 — Summary, per-mode stats and trend in one database round trip.

REAL GOTCHA

`Interview.find({ userId: "abc..." })` works because Mongoose casts the string to an ObjectId. `aggregate()` does **not** cast — the `$match` silently matched 0 documents until `new mongoose.Types.ObjectId(userId)` was used.

Frontend: the chart (recharts) is in its own lazy chunk (`ScoreTrendChart.jsx`); analytics are hidden for users with 0 completed interviews and hidden silently if the stats call fails, so the history list still works.

F12 Credits ledger

`User.credit` is the balance. The `Transaction` collection is the **passbook**: one row for every + or −, with the balance after it. It answers "where did my credits go?".

Type	Amount	Written by
`signup_bonus`	+500	Google sign-up or email register (unique per user)
`interview`	−50	`chargeCredits` at Start
`refund`	+50	Start failed after charging, or cleanup of a stuck start
`purchase`	+500 / 1500 / 4000	`fulfillOrder` after payment (unique `paymentId`)

ONE RULE: BALANCE AND LEDGER CHANGE TOGETHER

Every credit change goes through `chargeCredits` or `addCredits`, which update the user **and** insert the ledger row in one MongoDB transaction. A crash in between can't leave a balance without its row. The Redis cache for the user is cleared **after** the commit — clearing it inside the transaction could let another request re-cache the old balance before the commit.

WHY TRANSACTIONS NEED ATLAS / A REPLICA SET

MongoDB transactions only work on a replica set. Atlas is one. The tests use `MongoMemoryReplSet` for the same reason.

The account page shows the ledger newest-first with "Load more" (`GET /api/credits/transactions`, 10 per page, index `{ userId, createdAt: -1 }`).

KNOWN GAP

Users created before the ledger existed have no row for their old credits — their balance is right, the history just starts later.

F13 Payments with Razorpay

Plans: **Starter** 500 credits ₹99 · **Pro** 1500 ₹249 · **Max** 4000 ₹599. Two independent paths can confirm a payment — the browser (*verify*) and Razorpay's server (*webhook*) — and both must credit exactly once.

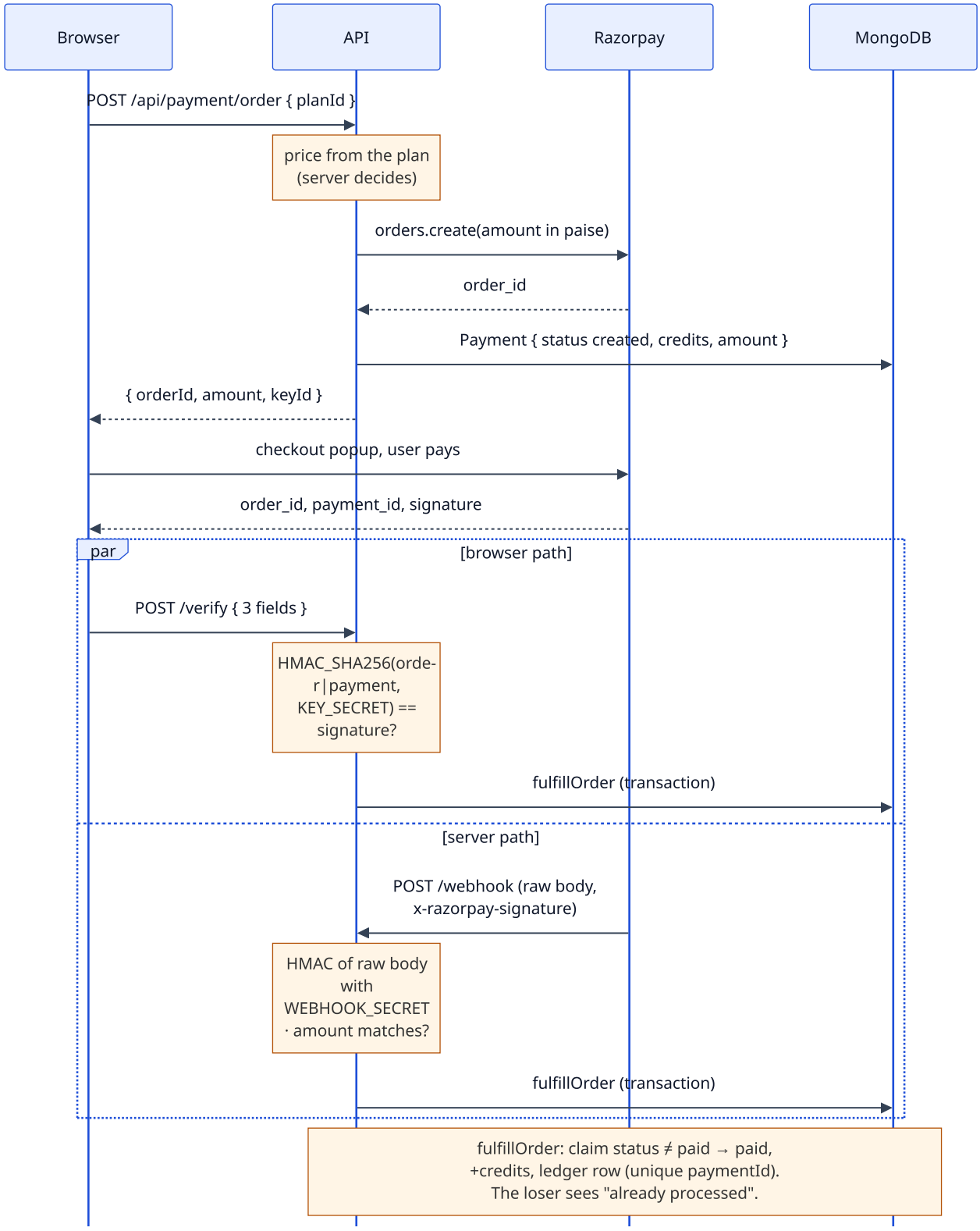


Figure F13.1 — Order, checkout, then verify and webhook racing safely.

server/src/modules/payment/payment.service.js (simplified)

```
const fulfillOrder = (orderId, paymentId) => mongoose.connection.transaction(async (session) => {
    const payment = await Payment.findOneAndUpdate(
        { razorpayOrderId: orderId, status: { $ne: "paid" } },          // claim
        { $set: { status: "paid", razorpayPaymentId: paymentId, paidAt: new Date() } },
        { returnDocument: "after", session },
    );
    if (!payment) return { alreadyProcessed: true };
    const user = await User.findOneAndUpdate({ _id: payment.userId }, { $inc: { credit:
payment.credits } }, { returnDocument: "after", session });
    await Transaction.create([{ userId: payment.userId, type: "purchase", amount: payment.credits,
balanceAfter: user.credit, paymentId }], { session });
    return { alreadyProcessed: false, credit: user.credit };
});
```

Security checklist

- **Server sets the price** — the client sends only `planId` (zod enum). Otherwise someone could buy 4000 credits for ₹1.
- **HMAC signatures** with secrets only Razorpay and our server know; compared with `crypto.timingSafeEqual` (a normal `===` stops at the first wrong character, which leaks timing).
- **Webhook uses the raw body** — mounted before `express.json()`.
- **Order must belong to the user** on verify (else 404); webhook checks `amount` equals what we asked for.
- `payment.failed` only marks `created` orders as failed, never a paid one.
- No keys configured → payments return 503 `PAYMENTS_DISABLED`, the rest of the app works.

> **WHY THE WEBHOOK MATTERS**
>
> The user may pay and close the tab before the browser calls verify. The webhook is Razorpay's server telling ours directly, so credits still arrive. Razorpay retries webhooks, which is why fulfilment must be idempotent.

F14 The AI client

One file, `server/src/lib/ai/openRouter.js`, talks to the LLM. Live we use **Groq** (`AI_PROVIDER=groq`). OpenRouter is kept only as a backup — switching back is an env change (`AI_PROVIDER=openrouter`), no code change.

server/src/lib/ai/openRouter.js

```
// both APIs are OpenAI-compatible — only URL, key and model differ
const PROVIDERS = {
    openrouter: {
        url: "https://openrouter.ai/api/v1/chat/completions",
        apiKey: env.OPENROUTER_API_KEY,
        models: [env.OPENROUTER_MODEL, ...env.OPENROUTER_FALLBACK_MODELS], // backup list
    },
    groq: {
        url: "https://api.groq.com/openai/v1/chat/completions",
        apiKey: env.GROQ_API_KEY,
        models: [env.GROQ_MODEL],                                  // one model
    },
};
const { url: AI_URL, apiKey: AI_KEY, models: MODELS } = PROVIDERS[env.AI_PROVIDER];

// request body: one model → { model }, several → { models } (OpenRouter's fallback feature)
{ ...(MODELS.length > 1 ? { models: MODELS } : { model: MODELS[0] }), messages,
  ...(json && { response_format: { type: "json_object" } }) }
```

Robustness inside `askAi` / `askAiJson`

Problem seen with real models	Handling
Reply wrapped in ```` ```json … ``` ```` or with extra text	`extractJson` takes from the first `{` to the last `}`.
HTTP 200 but an error in the body (all models busy)	Turned into a real error.
Empty content	Error that includes the model name and `finish_reason`.
Wrong shape / out-of-range numbers	zod schema; failure counts as a failed attempt.
Temporary failure	Default 2 retries with backoff 1 s, 2 s + random jitter (0–300 ms). Jobs pass `retries: 0` because BullMQ retries.
Slow provider	60 s axios timeout.

WHY JITTER?

If 100 users fail at the same second and all retry after exactly 1 s, they hit the provider together again. A small random extra delay spreads them out.

REAL MIGRATION STORY

Reports were slow on a free OpenRouter reasoning model (~3.4 s for a tiny prompt, often busy; backups failing). Switched to Groq with a provider map. The first model name tried no longer existed on Groq (404) — the fix was to list the account's models and benchmark them: gpt-oss-120b 1.4 s, gpt-oss-20b 1.2 s, qwen 0.8 s → chose `openai/gpt-oss-120b` for quality. Lesson: model catalogues change; verify, don't assume.

F15 Background queue and cleanup

BullMQ on Redis. Two queues: `ai` (resume + report) and `maintenance` (cleanup every 60 s).

Adding a job — with a per-user lock

server/src/lib/queue/aiJobs.js

```
export const enqueueAiJob = async (name, userId, data) => {
    if (!isRedisReady()) throw queueDown();                          // 503
    const jobId = randomUUID();
    // NX = only if the key doesn't exist → two parallel requests: only one wins
    const locked = await redis.set(lockKey(userId), jobId, "EX", 300, "NX");
    if (!locked) throw new AppError(429, "You already have an AI task running...", "AI_JOB_RUNNING");
    const reqId = getLogger().bindings?.().reqId;                     // logs of API + worker share
an ID
    await aiQueue.add(name, { ...data, userId: String(userId), reqId }, { jobId });
    return jobId;
};

// release only OUR lock — GET + DEL as one atomic Lua script
const RELEASE_LOCK_SCRIPT = `if redis.call("GET", KEYS[1]) == ARGV[1] then return redis.call("DEL",
KEYS[1]) end return 0`;
```

Processing a job

- `processAiJob` finds the handler by job name, runs it inside a logger that carries `jobId`, `reqId` and the attempt number.
- On success: release the lock, return the result (BullMQ stores it as `returnvalue`, kept 1 h).
- On failure: release the lock only on the **final** attempt. Only our own messages (`AppError`) reach the user; internal errors become "Something went wrong" (the real error is kept as `cause` in the logs).
- `GET /api/jobs/:id` maps BullMQ's many states to three: `pending`, `completed` (+result), `failed` (+message). Another user's job → 404.

Cleanup job (self-healing)

Finds	Does
`starting`, not updated for 10 min (max 100 per run)	Refund if the ledger has a charge and no refund yet → then delete. Refund **before** delete: a crash in between is retried safely next minute.
`finishing`, not updated for 15 min	Back to `in_progress`, so the user can Finish again.

The schedule is created with `upsertJobScheduler`, so starting several workers still gives **one** schedule, and the maintenance worker has concurrency 1.

DETAILS THAT MATTER IN PRODUCTION

- The worker needs its **own** Redis connection with `maxRetriesPerRequest: null` (it blocks waiting for jobs; BullMQ requires this).
- Redis must use `noeviction`, or jobs could be silently deleted when memory is full.
- Graceful shutdown waits up to 25 s for running jobs; unfinished ones become "stalled" and are re-run.

F16 Redis: rate limits and cache

Rate limits

Limiter	Limit	Key	Where
`globalLimiter`	100 / min	IP	every route (after `/health`)
`authLimiter`	10 / 15 min	IP	register, login, resend, forgot, reset
`aiLimiter`	10 / min	**userId**	resume, start, answer, finish (after `isAuth`)
`voiceLimiter`	20 / min	**userId**	`/api/transcribe` (Whisper)
`ttsLimiter`	40 / min	**userId**	`/api/tts` (Orpheus, one call per sentence)

Counters live in Redis (`rl:global:`, `rl:auth:`, `rl:ai:` prefixes), so every server copy shares the same count and a restart doesn't reset it. The AI limiter counts per **user**, so students on one college/hostel IP don't block each other.

Cache-aside

`server/src/lib/cache.js`

```
export const cached = async (key, ttlSeconds, load) => {
    if (!isRedisReady()) return load();                    // Redis down → just use the DB
    try {
        const hit = await redis.get(key);
        if (hit) return JSON.parse(hit);                     // HIT
    } catch (error) { /* log and fall through */ }
    const value = await load();                            // MISS → DB
    redis.set(key, JSON.stringify(value), "EX", ttlSeconds).catch(...); // no await: don't make the
user wait
    return value;
};
```

Key	TTL	Cleared when
`user:<id>`	60 s	credits charged/added, payment, email verified, password reset, Google login fixes an account
`stats:<id>`	10 min	an interview completes

FAIL OPEN — AND WHY /READY IGNORES REDIS

If Redis goes down, limiters let requests through (`passOnStoreError`) and the cache reads MongoDB. A broken helper should not take the whole app down. That's also why `/ready` reports Redis but doesn't fail on it: all servers share one Redis, so failing readiness would mark *every* server unhealthy at once.

BUT REDIS IS REQUIRED AT STARTUP

The rate-limit store loads its Lua scripts when the app starts. If Redis is missing then, the store stays broken forever — so the server refuses to start without Redis (later outages are tolerated). Also only one "Redis down" log line is written, not one per request.

F17 Config, validation and errors

Environment: fail fast

`config/env.js` validates `process.env` with zod when the server starts. A missing or wrong value prints a list and exits — much better than crashing on the first user.

```
// a few rules from env.js
PORT: z.coerce.number().int().positive().default(8000),           // "8000" → 8000
JWT_SECRET: z.string().min(16),
AI_PROVIDER: z.enum(["openrouter", "groq"]).default("openrouter"),
RUN_WORKER_IN_API: z.enum(["true", "false"]).default("false").transform((v) => v === "true"),
// .superRefine: the chosen provider's key is required; in production SMTP is required and CLIENT_URL
must be https
```

WHY NOT Z.COERCE.BOOLEAN()?

`z.coerce.boolean()` uses JavaScript truthiness: the string `"false"` is non-empty, so it becomes `true`. The enum + transform makes "false" really false.

Request validation

server/src/middlewares/validate.js

```
const validate = (schemas) => (req, res, next) => {
    for (const part of ["body", "params", "query"]) {
        if (!schemas[part]) continue;
        const result = schemas[part].safeParse(req[part] ?? {});
        if (!result.success) throw new AppError(400, /* "body.email: Enter a valid email; ..." */);
        // Express 5: req.query is a getter → replace with defineProperty
        Object.defineProperty(req, part, { value: result.data, writable: true, enumerable: true,
configurable: true });
    }
    next();
};
```

After validation the controller gets **clean** data: trimmed, lower-cased emails, numbers instead of strings, unknown fields removed. On the client, `getErrorMessage()` removes the `body.email:` prefix before showing the message.

Errors

- `AppError(status, message, code)` — our message is safe to show.
- Any other error → 500 "Internal server error"; the real error goes to the log with the stack.
- Every error response includes `requestId`.
- Status codes used: 400 bad input, 401 not logged in, 402 not enough credits, 403 email not verified, 404 not found / not yours, 409 conflict (already answered, finishing…), 429 limits, 502 email/payment provider, 503 AI/queue busy.

F18 Logging, monitoring, health

Request IDs everywhere

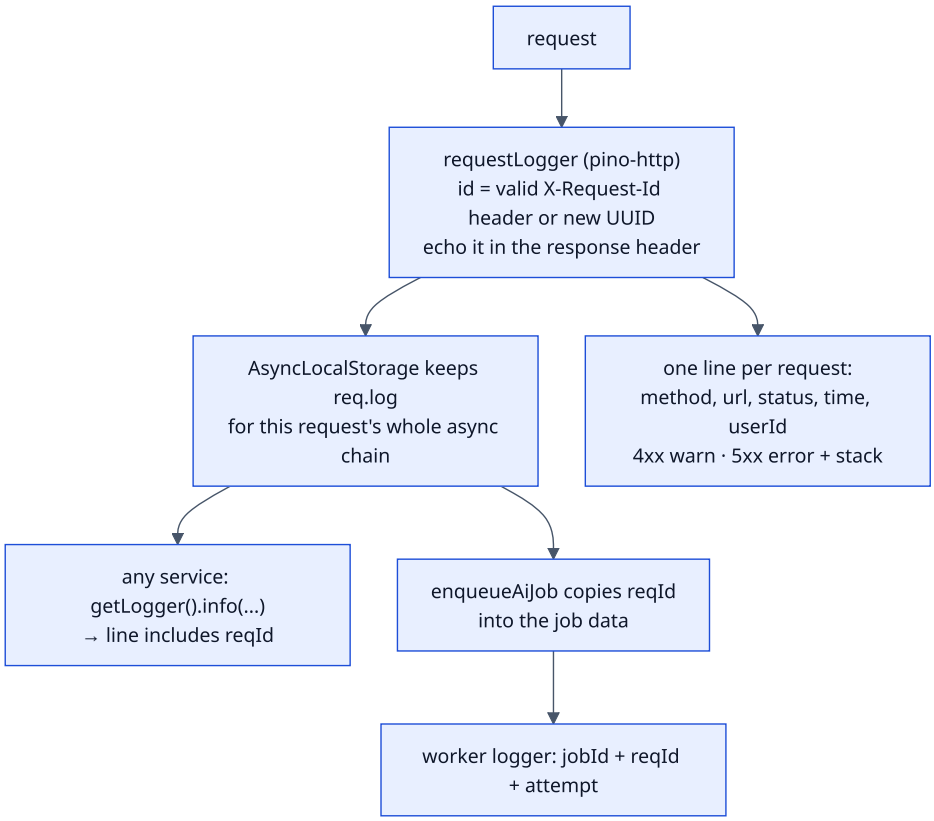


Figure F18.1 — One ID follows a request from the API into the background worker.

Feature	Detail
pino	JSON lines in production (fast, searchable), coloured pretty lines in development.
Redaction	Cookie, authorization and set-cookie headers never reach the logs.
Quiet health checks	`/health` and `/ready` are not logged (they run every few seconds).
Safe incoming ID	Only accepted if 1–64 chars of `[\w-]`, so nobody fills logs with a 1 MB header.
Sentry (optional)	Server and client; off when no DSN. Server sends only 5xx; client Sentry is a separate lazy chunk.
`GET /health`	"Process is alive" → `{ status: "ok" }`. Used by Render's health check.
`GET /ready`	"Can serve requests": checks MongoDB `readyState` first, then a ping → 200 or 503; reports Redis ok/down.

REAL BUG: EVERY REQUEST GOT THE ID "UNDEFINED"

`/^[\w-]{1,64}$/.test(undefined)` is **true** — `undefined` is converted to the string "undefined", which matches. Requests without the header got id "undefined", setHeader then threw and every request returned 500. Fix: check `typeof incoming === "string"` first.

F19 Tests, CI and deployment

How the server tests work

- **Vitest + Supertest**: real HTTP requests against the Express `app` (no `listen` needed — that's why `app.js` and `server.js` are separate).
- **MongoMemoryReplSet**: a temporary MongoDB in RAM, as a replica set because credits and payments use transactions.
- **Redis DB 1**: your dev data in DB 0 is never touched; everything is wiped before each test.
- **AI and email are mocked** with `vi.mock`; tests never read the real `.env`.
- Test files run one after another (`fileParallelism: false`) because they share one DB.

The race test, in words (server/test/interview.test.js)

```
user has 50 credits (enough for ONE interview)
fire 10 POST /start requests at the same time, each with a different requestId
expect: exactly 1 × 201 and 9 × 402
expect: credit = 0, and exactly one "interview" ledger row
```

Client tests

Testing Library + jsdom with the real Redux slices (`renderWithProviders`): the setup form (role required, not enough credits, trimmed values + UUID sent, errors shown, resume chips, non-PDF refused), the report page, `waitForJob` polling with fake timers, the voice picker / text splitter, and both voice hooks on their server paths (fake `MediaRecorder` and `audio.play`).

CI

`.github/workflows/ci.yml` runs on every push and PR: Node 22, a Redis 7 service container, npm cache, cached MongoDB binary → Prettier check → ESLint → server tests → client tests → client build. Older runs on the same branch are cancelled.

Deployment checklist (what actually had to be fixed to go live)

Problem on Render/Vercel	Fix
Server crashed: pino-pretty is a dev dependency	Set `NODE_ENV=production` (and `NODE_VERSION=22`).
Startup refused	`CLIENT_URL` must be the https Vercel URL.
Redis unreachable	`REDIS_URL` still pointed at localhost → use the Key Value internal URL.
API 404s through Vercel	The URL in `vercel.json` had been broken across two lines by a web editor.
Google login blocked	Add the Vercel domain to Firebase "Authorized domains".
Reports never finished	`RUN_WORKER_IN_API=true` was missing (service made by hand, not from `render.yaml`).

Slow / busy AI	Switched to Groq (`AI_PROVIDER` , `GROQ_API_KEY` , `GROQ_MODEL`).

SAY IT IN AN INTERVIEW

"Most deployment bugs were configuration, not code. That's why the server validates its whole environment with zod at startup and refuses to boot with an insecure or incomplete config — and why I'd add an alert if the queue has jobs but no worker."